

Manual Técnico — HuellitasEnCasa

1. Stack y versiones (fijas, sin `+latest`)

Componente	Versión
Kotlin	2.0.21
Android Gradle Plugin	8.5.2
Gradle	8.7 (wrapper)
KSP	2.0.21-1.0.28
Compose BOM	2024.09.00
Material3	1.3.0
Navigation Compose	2.8.0
Room	2.6.1
Lifecycle (ViewModel/runtime)	2.8.4
Coroutines	1.8.1
JDK	17
compileSdk / targetSdk	34
minSdk	24

Todas las versiones están fijadas explícitamente en `app/build.gradle.kts` y en los plugins del `build.gradle.kts` raíz.

2. Arquitectura

MVVM + Repository, en tres capas dentro de `app/src/main/java/com/educalab/huellitasencasa/`:

```

data/
  local/
    entity/      21 @Entity (20 del listado BASE + AcademyLessonProgressEntity, documentada)
    dao/          13 interfaces @Dao
    seed/         Contenido semilla (especies, alimentos, escenarios, misiones, insignias...)
                  HuellitasDatabase.kt
    repository/   ProfileRepository, PetRepository, ContentRepository, CareLogRepository,
                  ProgressRepository – envuelven los DAO y aplican reglas de domain/logic
  domain/
    model/        Enums compartidos + AcademyCurriculum (contenido curricular estático)
    logic/         CareEngine, WellbeingCalculator, MissionEngine, PlannerValidator,
                  ScenarioGrader, ProgressStateResolver – Kotlin puro, sin dependencias Android
  ui/
    theme/         Color.kt, Type.kt, Theme.kt (Material3)
    components/    Componentes reutilizables (barras, chips, tarjetas, drag&drop, ilustración)
    navigation/    Destinations.kt, HuellitasNavGraph.kt
    screens/       12 pantallas, cada una con su ViewModel co-localizado
    SessionViewModel.kt Estado de sesión (perfil/mascota activos), con alcance de Activity

```

Ningún Composable ejecuta SQL directamente ni contiene reglas de negocio complejas: los ViewModel llaman a los repositorios, que a su vez usan `domain/logic` para calcular y luego persisten con Room.

3. Base de datos (Room)

Ver docs/BASE_DE_DATOS.md para el modelo completo con DER. Puntos clave: - Base de datos única `huellitas_en_casa.db`, versión de esquema 1, `exportSchema = true`. - Claves foráneas con `onDelete = CASCADE` (o `SET_NULL` para recompensas de misión) para mantener integridad referencial real. - Índices únicos para evitar duplicados lógicos (p. ej. (`user_profile_id`, `mission_id`) en `mission_completions`, (`species_id`, `need_type`) en `pet_need_definitions`). - `SeedProvider.seedIfNeeded()` puebla la base de datos una única vez (comprueba `petSpeciesDao().count() > 0`), resolviendo referencias por código (p. ej. "PERRO" → `id` real) tras cada inserción, nunca asumiendo IDs fijos.

4. Lógica de dominio testeable

Todo en `domain/logic/` es Kotlin puro (sin `android.*`, sin Room, sin Compose), lo que permite testear reglas de negocio en JVM puro, sin emulador:

- **CareEngine**: clamp de indicadores a `[0, 100]`, aplicación de acciones, descenso de sesión con piso protector (`PROTECTED_FLOOR = 15`) para que la mascota nunca quede en estado crítico solo por el paso del tiempo.
- **WellbeingCalculator**: media de los 6 indicadores, niveles (`EXCELENTE/BIEN/NECESITA_ATENCION/ATENCION_URGENTE`), indicador(es) más bajo(s) para sugerir actividad.
- **MissionEngine**: aplica un evento real ("`ALIMENTACION`", "`HIGIENE`"...) al progreso de todas las misiones de ese tipo aún no completadas, capando en el objetivo y marcando `completed` al alcanzarlo.
- **PlannerValidator**: valida cada tarjeta de acción contra las franjas horarias permitidas para ese tipo de acción; un plan se aprueba con $\geq 80\%$ de aciertos.

- **ScenarioGrader**: compara índice de opción elegida contra el índice correcto.
- **ProgressStateResolver**: resuelve el estado visual (bloqueado/disponible/iniciado/completado/dominado) de un módulo a partir de intentos y aciertos reales.

5. Repositorios

Repositorio	Responsabilidad
<code>ProfileRepository</code>	Crear/leer/actualizar el perfil de cuidador
<code>PetRepository</code>	Especies, mascota activa, aplicar acciones de cuidado y descenso de sesión
<code>ContentRepository</code>	Tarjetas de alimentos, objetos del hogar, escenarios de bienestar + registro de intentos
<code>CareLogRepository</code>	Historial de acciones, sesiones, planificador diario
<code>ProgressRepository</code>	Misiones, insignias, decoraciones, progreso de Academia; <code>registerEvent()</code> es el punto único de entrada para progresar misiones y otorgar recompensas

6. ViewModels

Cada pantalla tiene su propio ViewModel (p. ej. `HubViewModel`, `FeedingViewModel`, `PlannerViewModel...`), construido con `AppViewModelFactory` (fábrica manual sencilla que inyecta los repositorios desde `HuellitasApplication`; no se usa Hilt/Dagger para mantener la complejidad proporcional al proyecto). El estado se expone como `StateFlow` y se consume con `collectAsState()` en Compose.

`SessionViewModel` vive con alcance de `Activity` (se obtiene con `viewModel()` sin `NavBackStackEntry` propio) y guarda el `profileId/petId` activos mientras la app está abierta.

7. Permisos y manifiesto

`AndroidManifest.xml` no declara ningún `<uses-permission>`: no hay INTERNET, cámara, micrófono, almacenamiento externo ni ubicación, porque ninguna función de la app los requiere.

8. Compilación

```
./gradlew clean
./gradlew testDebugUnitTest
./gradlew lintDebug
./gradlew assembleDebug
```

Requiere acceso a `google()` y `mavenCentral()` (Android Gradle Plugin, AndroidX, Compose, Room) y al Android SDK (compileSdk 34). Ver `docs/BUILD_REPORT.md` para el estado real de verificación en el entorno de generación de este proyecto.

9. Pruebas

`app/src/test/java/.../domain/` contiene 43 tests JVM puros sobre `domain/logic` y `domain/model` (CareEngine, WellbeingCalculator, MissionEngine, PlannerValidator, ScenarioGrader, ProgressStateResolver, AcademyCurriculum), **compilados y ejecutados realmente** en este entorno con un compilador Kotlin y JUnit 4.13.2 instalados localmente (sin necesidad de Android SDK, porque no dependen de `android.*`).

`app/src/test/java/.../data/` contiene 12 tests adicionales con Room en memoria + Robolectric (SeedProviderTest, MissionRewardFlowTest) que verifican persistencia real, idempotencia del seed y el flujo completo de desbloqueo de recompensas. Estos requieren Android SDK/Robolectric y **no se ejecutaron en este sandbox**; están listos para correr con `./gradlew testDebugUnitTest` en un entorno con Android Studio.

10. Mantenimiento y ampliación

- **Añadir una especie nueva:** agregar una entrada a `SeedSpecies.speciesList`, sus `needDefinitionsFor(...)`, y opcionalmente tarjetas/escenarios/objetos específicos con su `speciesCode`. Añadir su rama en `PetIllustration.paletteFor()` y `drawSpeciesAppendages()`.
- **Añadir una lección:** agregar un `AcademyLesson` a `AcademyCurriculum.lessons`; si se quiere que cuente para la misión "Academia de oro", actualizar `targetCount` en `SeedMissions` si el número total de lecciones cambia.
- **Añadir una misión:** agregar una entrada en `SeedMissions`, referenciando (opcionalmente) una insignia/decoración existente por código; `ProgressRepository.registerEvent()` la recogerá automáticamente por su `type`.
- **Cambiar el esquema de Room:** incrementar `version` en `@Database` y añadir una `Migration` explícita (no se usan migraciones destructivas en producción).

11. Ausencia deliberada de dependencias

No se usa Hilt/Dagger (fábrica manual suficiente para el tamaño del proyecto), no se usa Retrofit/OkHttp (no hay red), no se usa Firebase ni ninguna librería de analítica/anuncios, no se usa DataStore para datos sensibles (todo vive en Room). Esto mantiene la superficie de dependencias mínima y coherente

con el requisito de app 100% offline y sin tracking.